counter — CLI Reference

The Bitcoin Counters command-line tool: wallet, minting, reads, and daemons.

Companion to the Counterparty Inscriptions build reference · working draft · June 2026

> **counter** is the command-line tool for Counterparty Inscriptions (Bitcoin Counters). It drives named Bitcoin Core wallets, mints counters from files, reads counters by number or asset, and runs the indexer and content server. A single binary, `counter`, with two top-level shapes: wallet-scoped commands (`counter wallet --name <w> …`) and global commands (`counter info` , `counter index` , …).

1. What counter talks to

counter depends on **two external programs** and produces its **own state**. It reimplements neither backend.

External backend	Used for
Bitcoin Core (RPC, `txindex=1`) — **including its wallet**	Holds keys, funds and signs transactions, tracks UTXOs; broadcasts the commit and reveal; reads blocks and witnesses; BTC balances.
Counterparty Core v2 (REST API)	Composing the issuance and send messages, resolving `asset_id` / names, and reading asset balances, XCP balance, and issuance validity.

counter also produces its own state: **the index DB** — written by `counter index` and read by `server` , `info` , and `list` (counter numbers, content, listings, ownership rollups). It keeps no persistent mint state — an inscribe broadcasts both transactions at once (§3), so there is nothing to resume.

> Counterparty's legacy composing CLI (`counterparty-client`) is unsupported, so counter composes Counterparty messages through the **Core v2 REST API**, then assembles the taproot commit/reveal transactions itself. Critically, the issuance is composed with **OP_RETURN encoding (not Counterparty's taproot-envelope mode)**, so the witness is left free for counter's own `COUNT` envelope (see the build reference, §11).

2. Command tree

```
counter wallet create   --name <w>          # prints the seed phrase once
counter wallet restore  --name <w>
counter wallet --name <w> receive
counter wallet --name <w> balance            # BTC + XCP + counters
counter wallet --name <w> inscriptions       # counter holdings by asset + qty
counter wallet --name <w> inscribe --file <path>
                [--asset NAME] [--fee-rate R] [--commit-fee-rate R]
                [--destination ADDR] [--supply N] [--divisible] [--dry-run]
counter wallet --name <w> send <address> <asset> <amount>

counter info <number|asset> [--save PATH | --raw | --json]
counter list [--recent N | --owner ADDR | --block RANGE]
counter validate <txid>
counter status

counter index
counter server
```

3. Wallet commands

Lifecycle

Wallets are **Bitcoin Core wallets** — like `ord` , counter generates a seed at `create` , prints it **once**, imports the derived descriptors into Core, and from then on drives Core over RPC. `--name <w>` selects the Core wallet (equivalent to `bitcoin-cli -rpcwallet=<w>`); Core stores keys, signs, and tracks UTXOs. There is no standalone `seed` command — the phrase is shown only at creation, so write it down.

Command	Action

`wallet create --name <w>`	Generate a seed, derive descriptors, import them into the Core wallet `<w>`, and **print the seed phrase once**. Wraps `createwallet` + `importdescriptors`.
`wallet restore --name <w>`	Recreate the wallet from that seed phrase (re-derives descriptors and imports into Core).

> **Wallet type (v1):** a single descriptor wallet, **taproot only**. `create` makes a blank descriptor wallet and imports BIP86 `tr(…)` descriptors derived from the seed (receive + change), so every wallet address is `bc1p`. One address type keeps the source, funding, commit, and reveal uniform. segwit-v0 (BIP84) can be added later only to spend XCP already held at a `bc1q` address.

Funds & holdings

Command	Action
`… receive`	Show a receive address (Core's `getnewaddress`) for BTC, XCP, or assets.
`… balance`	Show **BTC, XCP, and counter holdings** together — BTC from Core, XCP and counters from Counterparty v2. XCP is shown because minting a named asset costs 0.5 XCP.
`… inscriptions`	Counter holdings only, grouped by Counterparty asset name with quantity, e.g. `1 ZOMBIEPEPES` `0.0003131 LORDKEK` `42 A123441413414`
`… send <addr> <asset> <amount>`	Transfer a counter (a Counterparty asset): compose the send via Counterparty v2, fund and sign via Core. Optionally `--btc` to send BTC instead.

Keep `balance`'s counter section and `inscriptions` rendering identical so the two views never drift; `inscriptions` is just the focused, no-BTC-noise version.

> **Taproot wallet, any payee.** The wallet is taproot-only, but `send <address>` and `inscribe --destination` accept **any** address type (legacy, P2SH, segwit-v0, or taproot) — an output's type is independent of the wallet's inputs. `receive` always returns a `bc1p` address (one counter must control). Only an **asset** destination must be a type Counterparty recognises as a holder (legacy / v0 / taproot, post-v11); a plain BTC send goes anywhere.

inscribe

The headline command: mint a counter from a file. By default it mints a **free numeric asset**; `--asset NAME` makes it a named asset (which costs 0.5 XCP), and a dotted name is detected as a subasset.

```
counter wallet --name abc inscribe --file cat.png                    # free numeric  (A123...)
counter wallet --name abc inscribe --file cat.png --asset ZOMBIEPEPES  # named (0.5 XCP)
counter wallet --name abc inscribe --file cat.png --asset PEPECASH.Z   # subasset
counter wallet --name abc inscribe --file cat.png --commit-fee-rate 2 --dry-run  # build only; print raw hex
```

Flag	Meaning
`--file PATH`	The file to inscribe (required).
`--asset NAME`	Mint a named asset (or `PARENT.CHILD` subasset). Omit for a free numeric asset.
`--fee-rate R`	Fee rate (sat/vByte) for the **reveal** transaction.
`--commit-fee-rate R`	Fee rate (sat/vByte) for the **commit** transaction. Defaults to `--fee-rate` if omitted.
`--destination ADDR`	Address to receive the minted counter (the Counterparty asset). Defaults to the issuing wallet.
`--supply N`	Issued quantity (default 1).
`--divisible`	Make the asset divisible (default indivisible — a 1-of-1).
`--dry-run`	Build and sign both transactions but do **not** broadcast; print the **combined cost** (both transactions' fees + 0.5 XCP if

	named) and output the commit and reveal raw hex for inspection or manual broadcast.

> **inscribe always mints a new asset.** A counter binds to an asset's *first* issuance, so you cannot inscribe onto an asset that already exists. `--asset ZOMBIEPEPES` fails cleanly if that name is already registered. Named assets also require the wallet to hold **0.5 XCP**; numeric assets need only BTC.

The inscribe flow (commit + reveal)

An inscribe is two transactions, but a single uninterrupted action — the same approach `ord` uses. counter builds and signs **both** transactions up front, then broadcasts them back-to-back. The reveal spends the commit's output as a **child transaction**, so it sits in the mempool on top of the still-unconfirmed commit (CPFP). There is **no confirmation wait between them**, and no `resume` command.

```
1. BUILD      build the "COUNT" envelope from the file; derive the P2TR
              commit address; build AND sign both transactions:
                commit = fund the P2TR commit output (from the Core wallet)
                reveal = vin[0] funded wallet UTXO (XCP source)
                       + an input spending the commit output (reveals the
                         "COUNT" envelope);
                         OP_RETURN issuance composed via Counterparty v2;
                         signed by Core
2. BROADCAST  send the commit, then immediately the reveal (a child of the
              unconfirmed commit). Both enter the mempool and confirm together.
3. DONE       on confirmation, Counterparty processes the issuance and the
              counter is minted.
```

Because both transactions are signed before either is sent, the only failure window is the instant between the two broadcasts — and the reveal is already built, so it can simply be re-broadcast. No on-disk mint state is required. `--dry-run` stops after step 1: it prints the cost and outputs the commit and reveal raw hex, without sending anything.

4. Read commands

These are public and need no wallet.

Command	Action
`info <number|asset>`	Human-readable metadata: counter number, asset, current owner, MIME, size, mint txid, block.
`info <number|asset> --save PATH`	Write the counter's file to disk.
`info <number|asset> --raw`	Stream raw file bytes to stdout (for piping). This is what `server`'s `/content/<number>` endpoint serves.
`info <number|asset> --json`	Metadata as JSON (for scripts).
`list [--recent N | --owner ADDR | --block RANGE]`	List counters by recency, owner, or block range.
`validate <txid>`	Report whether a transaction is a valid counter, **and why or why not** (envelope present? issuance valid under Counterparty? first issuance? reserved asset?). The primary debugging tool, and the basis for test vectors.
`status`	Tips/health of all three backends: bitcoind height, Counterparty Core height, and the counter index height — so lag is visible at a glance.

Lookups accept either a counter number or a Counterparty asset name/longname, since both uniquely identify a counter.

5. Daemon commands

Command	Action
`index`	Build and continuously update the counters database: scan blocks from taproot activation (the earliest a counter can exist), parse `COUNT` envelopes, confirm the matching first issuance via Counterparty Core, assign counter numbers, store records. Sync only.
`server`	Serve the query + content API on top of the index (the endpoints behind `info`, `list`, and `/content/<number>`). Requires an index to read from.

Keeping `index` (writer) and `server` (reader) separate lets you run multiple read servers off one index, and makes it explicit that `server` is useless without a synced `index`.

6. Backend map (which command calls what)

Command	Bitcoin Core	Counterparty v2	Index (own)
inscribe	build, fund, sign, broadcast commit+reveal (Core wallet)	compose issuance (OP_RETURN); asset name check	—
send	fund, sign, broadcast	compose send	—
balance / inscriptions	BTC balance	XCP + asset balances	asset → counter mapping
receive	getnewaddress	—	—
info / list	—	owner lookup (current holder)	numbers, content, listings
validate	read tx/witness	issuance validity	—
status	tip	tip	tip
index	blocks + witnesses	issuance validity + asset data	writes
server	—	—	reads

7. Configuration & files

- **Config** — endpoints for the three backends (Bitcoin Core RPC URL + cookie/credentials, Counterparty Core v2 API URL, index DB path/URL), plus network (mainnet/testnet) and the indexer's scan-start height (taproot activation — the earliest a counter can exist; not a protocol constant). A single config file with per-backend sections.
- **Wallets** — Bitcoin Core wallets. `--name <w>` selects the Core wallet (`-rpcwallet=<w>`). `create` generates a seed, prints it once, and imports the descriptors into Core; `restore` re-imports from that seed. Core then holds the keys, signs, and tracks UTXOs — counter keeps no key material and no mint state.
- **Index DB** — the counters database written by `index` and read by `server` , `info` , and `list` .

8. Suggested build order

A minimal path that exercises every backend early:

1. `status` + `config` — wire up and health-check all three backends first.
2. `index` — the database and numbering; nothing reads without it.
3. `info` / `list` + `server` — the public read path and content endpoint.
4. `wallet create` / `receive` / `balance` — wallet basics.
5. `inscribe` (with `--dry-run` first, then the build-both / broadcast-both flow) — the hard part, built last on top of working reads.
6. `send` , `validate` — round out transfer and debugging.

Companion to the Counterparty Inscriptions build reference. counter owns transaction assembly, the commit/reveal inscribe flow, numbering, and content serving; key custody and signing live in Bitcoin Core; asset identity, ownership, issuance validity, and the XCP burn are delegated to Counterparty Core. Keep this layer thin.